Insurance AI Agent

1. Project Background: Current vs. Proposed System

1.1 The Current System (Traditional Limitations)

Traditional insurance portals and customer support workflows face significant structural hurdles: policy wordings, endorsement schedules, and waiting periods are locked within multi-page PDF documents or dense spreadsheet rate cards. Extracting accurate terms manually requires substantial human effort, while generic AI models frequently hallucinate terms without verified clause citations. Furthermore, customer intake is typically gated behind rigid, multi-page forms that lead to high abandonment rates.

1.2 The Proposed System (The Insurance AI Agent Platform)

The proposed system establishes an autonomous, 24/7 Insurance AI Agent designed to serve as an intelligent copilot for policyholders, advisors, and underwriters. Rather than forcing customers to navigate dense policy manuals or endure frustrating questionnaire forms, the agent conducts natural conversational advisory to discover personal and commercial coverage needs. It actively analyzes customer context—whether an entrepreneur seeking commercial liability, a parent safeguarding family health, or a homeowner protecting property—and delivers tailored plan recommendations with complete clarity on coverage limits, deductibles, waiting periods, and exclusions. Customers can converse freely, review details of their active enrolled policies, compare plans side-by-side, and bookmark favorite insurance options for ongoing evaluation—delivering an intuitive, transparent, and trustworthy insurance experience.

- **Insurance Domain Scope:** Commercial, Health, Auto, Home, Life, Accident, Agriculture, and Travel.
- **Primary LLM Engine:** Groq Cloud LPU (OpenAI GPT-OSS 120B / openai/gpt-oss-120b).
- **Data & Vector Knowledge Base:** MongoDB Database + Qdrant Vector Store.

Core Architectural Benchmarks

Metric / Benchmark Area	Implementation & Architectural Details
Vector Retrieval Engine	Qdrant Vector Store using fast cosine similarity search across 384-dimensional policy embeddings with insurance category filtering.
Generative LLM Engine	Groq Cloud LPU hardware running OpenAI GPT-OSS 120B (openai/gpt-oss-120b) with sub-second generation and strict JSON formatting.
Local Embedding Engine	Xenova/all-MiniLM-L6-v2 executed locally via @huggingface/transformers ONNX CPU runtime with zero external token cost.
Two-Layer Data Pipeline	Direct integration with MongoDB for transactional ground truth and Qdrant for semantic policy clause similarity.
Pre-RAG Ingestion Guardrails	5-stage verification including SHA-256 deduplication, 4-point stratified sampling, and Groq AI Gatekeeper validation.
Customer Intake Methodology	Natural language semantic discovery with dynamic slot extraction and zero rigid numbered checklist forms.
Insurance Domain Scope	Comprehensive multi-line coverage for Commercial, Health, Auto, Home, Life, Accident, Agriculture, and Travel.

2. Technology Stack Breakdown

The system architecture is structured across three clearly defined, easily understandable tiers:

2.1 Frontend Technology Stack

Component	Technology
SPA Framework	React
Build Engine	Vite
Design System	Tailwind CSS
Iconography	Lucide React
HTTP Client	Axios
Favorites Sync	Browser Custom Events

2.2 Backend Technology Stack

Component	Technology
Runtime Server	Node.js & Express
Transactional Database	MongoDB & Mongoose
File Ingestion	Multer (for PDF, Word DOCX, and text)
Workbook Parser	SheetJS (xlsx)
Document Parsers	pdf-parse & mammoth
Security & Auth	jsonwebtoken & bcryptjs

2.3 LLM & Vector AI Engine Stack

Component	Technology
Primary LLM Engine	Groq SDK (openai/gpt-oss-120b)
Vector Database	Qdrant Vector Store
Local Embeddings	all-MiniLM-L6-v2 (ONNX)
Intent Extractor	AI Gatekeeper Engine

3. Pre-RAG Ingestion Pipeline & Quality Guardrails

Uploaded documents undergo a streamlined five-stage verification pipeline before vector indexation:

- **Stage 1 — Cryptographic Hash:** Calculates SHA-256 on the raw file buffer to immediately detect duplicates, preventing redundant embeddings and database clutter.
- **Stage 2 — Multi-Format Text Extraction:** Converts PDF policy manuals, Word DOCX documents, multi-sheet Excel workbooks, and CSVs into standardized clean text streams.
- **Stage 3 — 4-Point Stratified Sampling:** Extracts text samples at 0%, 33%, 66%, and 90% document offsets, enabling fast policy evaluation in under 1,200 tokens.
- **Stage 4 — Domain Density Evaluation:** Scans for 12 statutory insurance terms (deductibles, excess, waiting periods, limits) to confirm document legitimacy and override false rejections.
- **Stage 5 — AI Gatekeeper Verification:** OpenAI GPT validates policy authenticity, confirms the insurance category, and generates a validation confidence score prior to vectorization.

Two-Layer Architecture & Simple Data Flow

The system organizes insurance data cleanly across two simple layers to prevent any plan confusion:

- **1. Database Storage (MongoDB):** Stores customer accounts, enrolled policies, and insurance plans safely and reliably.
- **2. Vector Search (Qdrant):** Searches policy documents in milliseconds so the AI can find the right terms instantly.
- **Independent Plan Records:** Each insurance plan is stored as its own separate record with its own vector, so policies never get mixed up.
- **Easy Policy Updates:** Updating or deleting one plan updates only that plan, without affecting any other policies in the catalog.

4. Conversational AI Advisor & Dynamic Intake

The conversational advisor guides policyholders through discovery, comparison, and enrollment using dynamic natural language dialog rather than rigid, checklist-style interrogations.

Core Conversational Principles

1. **Conversational Needs Discovery First:** When a user asks an exploratory inquiry (e.g. *'I opened a small bakery with my brother, what insurance do we need?'*), the advisor explains relevant coverage types (Public Liability, Commercial Property, Business Interruption) and asks clarifying questions regarding their activities and budget. Specific plan recommendation cards are held in reserve until the user's specific context is understood.
2. **Dynamic Semantic Slot Extraction:** The advisor extracts profile attributes semantically (whoIsInsured, insuredName, coveredAge, healthOrRiskConditions, insuranceType) directly from prose, adapting to the relevant category without rigid numbered forms.
3. **Progressive Ground Truth Engine:** As intake details are shared, saveProgressivePolicyToDb records a draft policy in MongoDB. The persistent ground truth is injected into subsequent prompts, guaranteeing that the AI never re-asks for details already provided.
4. **Multi-Person Separation:** If a policyholder inquires about coverage for multiple family members (e.g. Father and Mother separately), the advisor maintains isolated records in extractedProfiles, preventing cross-contamination of medical history and age parameters.
5. **Enrolled Policy Catch-Up:** Users can ask *'What does my active policy cover?'* or *'Explain dad's plan'*. The advisor identifies the policy by its #POL-... number from database ground truth and thoroughly explains its limits, premiums, and benefits.

5. Favorites System, Security & REST API

Cross-Tab Real-Time Favorites Synchronization

To allow users to bookmark and evaluate insurance options across both the plan catalog and AI conversations, a unified Favorites subsystem is integrated across the frontend:

- **Heart Toggle Action:** Embedded on every plan card in InsurancePage.jsx and every AI recommendation card in AiChatPage.jsx. Turns vibrant red (fill-rose-500) when active.
- **Global Custom Event Sync:** Dispatches insurance_favorites_changed browser events on mutation, ensuring instant updates across all open tabs.
- **Profile Favorites Hub:** Accessible via the **Favorites ({count})** side tab in ProfilePage.jsx. Displays saved cards with coverage limits, premiums, deductibles, and direct 'Talk to Advisor ■' shortcuts.

Role-Based Access Control (RBAC) Matrix

Role Name	Authorized Personas	Operational Privileges & Boundaries
COMPANY_ADMIN	System Executive & Underwriting Lead	Full CRUD on users, batch ingestion, granular policy mutations, vector purges.
UNDERWRITER / EMPLOYEE	Claims Adjuster & Underwriter	Policy catalogue inspection, inline policy edits, customer claim reviews.
POLICYHOLDER	Consumer / Insured Client	AI Advisory chat, personal policy portfolio, KYC consent, favorites management.

Key REST API Endpoints

HTTP Method & Route	Primary Controller	Functional Description
POST /api/agent/query	ragAgent.controller.js	Unified advisory endpoint with semantic retrieval & intake persistence.
POST /api/policy-documents/upload	policyDocument.controller.js	Multi-file upload with AI Gatekeeper validation & disaggregation.
PUT /api/policy-offers/:id	policyOffer.controller.js	Mutates single policy terms and re-syncs Qdrant vector point.
DELETE /api/policy-offers/:id	policyOffer.controller.js	Permanently deletes offer from MongoDB and purges vector point in Qdrant.
GET /api/policies	policyDetail.controller.js	Retrieves all enrolled policies for the authenticated policyholder.

6. End-to-End System Data Flow Diagram

The following architecture diagram details the complete data lifecycle of the Insurance AI Agent platform—illustrating how documents are ingested, vectorized, stored, and queried through conversational RAG with progressive database synchronization:

STEP 1: CLIENT INGRESS & USER ACTION
Users & Portals: Policyholders explore coverage via AiChatPage.jsx; Underwriters upload manuals via UploadDocumentPage.jsx. **API Gateway:** Secure REST endpoints (Express 5 + JWT Auth) route requests to dedicated ingestion or query pipelines.

↓ **DUAL PROCESSING PATHWAYS (INGESTION vs. CONVERSATIONAL RAG)** ↓

PATH A: DOCUMENT INGESTION PIPELINE	PATH B: CONVERSATIONAL RAG QUERY PIPELINE
1. File Streaming: Multer streams PDF, DOCX, XLSX, and CSV documents. **2. Text Extraction:** pdf-parse and mammoth extract verbatim clause streams. **3. Semantic Chunking:** Partitions text into clauses with codes and pre-headers. **4. Local ONNX Embedder:** all-MiniLM-L6-v2 generates 384-dimensional dense vectors locally ($0 API cost). **5. Vector Upsert:** Stores points directly into Qdrant collection insurance_policy_rules.	**1. User Inquiry:** Customer sends question via AiChatPage.jsx. **2. Sliding Window:** Last 16 messages (8 turns) formatted with 400-char caps. **3. Semantic Retrieval:** Question is embedded and searched against Qdrant using cosine similarity in <15ms. **4. Ground Truth Ingestion:** Active policy details fetched from MongoDB to eliminate memory loss. **5. Unified LLM Call:** Groq Cloud LPU (openai/gpt-oss-120b) generates verified JSON answers.

↓ **STORAGE CONVERGENCE & AUTO-INTAKE SYNCHRONIZATION** ↓

STEP 3: PERSISTENT STORAGE & REAL-TIME CLIENT FEEDBACK
Qdrant Vector Database: Maintains 384-dim policy vectors with category payloads for sub-second similarity retrieval. **MongoDB Transactional Store:** Reliably saves user profiles, uploaded document metadata, and policy catalogs. **Progressive Auto-Intake Persistence:** savePolicyIntake writes draft policyholder details to MongoDB in real time. **Instant Live Client Sync:** UI updates dynamically with tailored plan cards, intake progress bars, and instant resumption.